Administration & Tuning des Bases de données

Gestion de l'audit et des connexions Oracle

Objectifs

- **Comprendre l'audit Oracle** : Apprendre à configurer l'audit des connexions pour enregistrer les événements de connexion/déconnexion.
- **Configurer l'audit des sessions** : Suivre les connexions réussies et échouées.
- **Analyser les logs d'audit** : Interroger les tables d'audit pour récupérer des informations.
- **Mettre en œuvre la politique d'audit** : Configurer des politiques d'audit adaptées à la gestion des connexions.

Qu'est-ce que l'audit Oracle ?

- **Audit Oracle** : Processus qui permet d'enregistrer les actions des utilisateurs sur la base de données.
- Permet de suivre les événements importants comme les connexions et les actions exécutées par les utilisateurs.
  - Suivi des tentatives de connexion.
  - Traçabilité des actions sensibles.
  - Détection des accès non autorisés.
  - Compliance avec des normes de sécurité.
- Fournit une visibilité sur la sécurité de la base de données et l'intégrité des données.

4

Types d'audit Oracle

- **Audit de session** :
  - Enregistre les événements de **connexion/déconnexion.**
- **Audit des actions** :
  - Surveille les actions spécifiques des utilisateurs comme les requêtes MLD (INSERT, UPDATE, DELETE).
- **Audit des privilèges** :
  - Permet de suivre les modifications des privilèges des utilisateurs.

5

Paramètre audit_trail

- Le paramètre **audit_trail** détermine où les événements d'audit sont enregistrés.
- Valeurs possibles :
  - **NONE** : Aucune trace d'audit.
  - **DB** : Enregistre les événements d'audit dans la table **DBA_AUDIT_TRAIL**.
  - **DB,EXTENDED** : Enregistre les événements d'audit avec des informations supplémentaires.
  - **OS** : Écrit les événements dans les fichiers journaux du système d'exploitation.

6

Configuration de d'audit de connexion

- **Activer l'audit** :
  - Utiliser la commande **ALTER SYSTEM** pour configurer l'audit.
  - Exemple pour activer l'audit dans la base de données :
    - SHOW PARAMETER AUDIT;
    - **ALTER SYSTEM SET audit_trail = 'DB' SCOPE = SPFILE;**

- **Auditer les connexions réussies** :
  - Utiliser la commande AUDIT SESSION :
    - **AUDIT SESSION WHENEVER SUCCESSFUL;**
- **Auditer les connexions échouées** :
  - Utiliser la commande AUDIT SESSION :
    - AUDIT SESSION WHENEVER NOT SUCCESSFUL;
- **Redémarrer Oracle si nécessaire** pour appliquer la modification :
  - **SHUTDOWN IMMEDIATE;**
  - **STARTUP;**

Visualisation des logs d'audit

- **Vérifier l'audit des connexions** :
  - Utiliser la vue **DBA_AUDIT_TRAIL** pour voir les connexions.
    - **SELECT** USERNAME, TIMESTAMP
      FROM DBA_AUDIT_TRAIL
      WHERE ACTION_NAME = 'LOGON';

- **USERNAME** : Nom d'utilisateur ayant effectué l'action.
- **TIMESTAMP** : Horodatage de l'action.
- **ACTION_NAME** : Type d'action (par exemple, LOGON,LOGOFF,
- **RETURNCODE** : Code de retour de l'action (0 pour succès, autre pour échec).

Visualisation des Logs d'Audit

- **Afficher les connexions réussies** :
  - SELECT USERNAME, TIMESTAMP
    FROM DBA_AUDIT_TRAIL
    WHERE ACTION_NAME = 'LOGON'
    AND TO_CHAR(TIMESTAMP, 'HH24') NOT BETWEEN 8 AND 18;
  - Cette requête montre les connexions réussies en dehors des heures de travail (8h-18h).

- **Vérifier les tentatives échouées** :
  - SELECT USERNAME, TIMESTAMP, ACTION_NAME, RETURNCODE
  - FROM DBA_AUDIT_TRAIL
  - WHERE ACTION_NAME = 'LOGON' AND RETURNCODE != 0;

Analyse des Résultats

- **Connexions réussies** :
  - Interrogez les logs pour identifier les utilisateurs qui se connectent.
  - Filtrez les données selon l'heure de connexion, la durée, ou l'adresse IP.
- **Connexions échouées** :
  - Vérifiez les raisons des connexions échouées, les tentatives incorrectes de mot de passe, etc.
  - Les erreurs peuvent être retournées dans RETURNCODE (par exemple, code 1017 pour mot de passe incorrect).

Code	Signification
1017	Mot de passe incorrect
28000	Nom d'utilisateur inconnu
12224	Compte verrouillé
1034	Base de données non disponibl
12514	Service TNS inconn

- **Vérifier les politiques d'audit** :
  - Utiliser la vue DBA_AUDIT_POLICIES.
    - **SELECT** *
      FROM DBA_AUDIT_POLICIES;

Exemple de requête de vérification

- **SELECT** USERNAME, TIMESTAMP
  FROM DBA_AUDIT_TRAIL
  WHERE ACTION_NAME = 'LOGON';

```
USERNAME                       TIMESTAM
------------------------------ --------
SYSTEM                         17/02/25
SYSTEM                         17/02/25
SYSTEM                         17/02/25
USER_NIT                       17/02/25

SQL> |
```


Tests de l'audit de connexion

- **Connexion avec un utilisateur** :
  - Connectez-vous à la base de données en tant qu'utilisateur et effectuez une connexion.
- **Exécutez la requête pour vérifier l'audit** :
  - Vérifiez dans la table **DBA_AUDIT_TRAIL** que la connexion est bien enregistrée.

Désactivation de l'audit

- **Désactiver l'audit de connexion** :
  - Après le test, vous pouvez désactiver l'audit avec la commande :
    - **NOAUDIT SESSION;**

- Une **politique d'audit Oracle** désigne un ensemble de règles et de configurations utilisées pour **surveiller, enregistrer** et **analyser les activités des utilisateurs** et **des administrateurs** dans une base de données Oracle.
- Elle permet de suivre les événements significatifs afin de garantir la sécurité, la conformité, et l'intégrité des données.

Les éléments clés d'une politique d'audit Oracle

- **Les éléments clés d'une politique d'audit Oracle incluent :**
  - **1. Types d'objets à auditer** : Définition des objets de la base de données à auditer, comme les tables, vues, utilisateurs, ou procédures stockées.
  - **2. Types d'événements à auditer :** Choisir quels événements seront audités, comme les connexions, les modifications de données (INSERT, UPDATE, DELETE), l'exécution de certaines commandes (GRANT, REVOKE, etc.), ou l'accès à des objets sensibles.
  - **3. Méthode d'audit :**
    - **Audit statique** : Audite des actions sur des objets spécifiques en fonction de configurations manuelles.
    - **Audit dynamique** : Audite automatiquement toutes les actions d'un certain type, indépendamment de l'objet ou de l'utilisateur.

Les éléments clés d'une politique d'audit Oracle

- **Les éléments clés d'une politique d'audit Oracle incluent :**
  - **4. Stockage des données d'audit** : Les informations sur les événements audités sont stockées dans des tables spéciales ou des fichiers journaux (comme **DBA_AUDIT_TRAIL** ou **AUD$**).
  - **5. Niveaux d'audit :** Selon les besoins, l'audit peut être configuré à différents niveaux :
    - **Global** : Auditer toutes les connexions ou toutes les actions.
    - **Spécifique** : Auditer des utilisateurs ou des actions spécifiques, comme les accès ou les modifications des données sensibles.
  - **6. Réaction aux événements:** Définir les mesures à prendre en cas d'événements suspects ou non autorisés, comme des alertes ou l'exécution de scripts automatiques.

19

Types de Politiques d'Audit

- **1. Audit global (Audit standard) :**
  - Audit des actions de connexion/déconnexion, modifications de schémas, changements de structures de données.
- **2. Audit des objets (Object audit) :**
  - Permet de suivre les actions sur des objets spécifiques tels que les tables, vues, procédures stockées.
- **3. Audit de session (Session audit) :**
  - Surveille les connexions réussies et échouées, et les actions exécutées pendant la session.

20

Types de Politiques d'Audit

- **4. Audit basé sur les privilèges (Privilege-based audit) :**
  - Audite l'utilisation de privilèges spécifiques (par exemple, les privilèges SYSDBA ou DBA).
- **5. Audit des erreurs (Error audit) :**
  - Permet de suivre les erreurs de connexion, les requêtes échouées, ou les actions non autorisées.

21

Configuration des politiques d'audit dans Oracle

- **Paramètre d'audit : AUDIT_TRAIL**
  - Permet de configurer la manière dont Oracle gère l'audit.
  - **Options :** DB, DB_EXTENDED, NONE.
- **Activation d'une politique d'audit (Audit de session) :**
  - **AUDIT SESSION;**
    - Cette commande enregistre les connexions et déconnexions des utilisateurs.
    - Peut être étendu avec des conditions spécifiques (ex : **WHENEVER SUCCESSFUL** ou **WHENEVER NOT SUCCESSFUL**).

22

L'Audit de Connexion – Politique de connexion

- Audit des connexions avec la politique AUDIT SESSION
- **Exemple de configuration :**
  - AUDIT SESSION WHENEVER SUCCESSFUL;
  - AUDIT SESSION WHENEVER NOT SUCCESSFUL;
- **But :** Enregistrer toutes les connexions réussies et échouées.
- **Résultats de l'audit :** Table DBA_AUDIT_SESSION
- **Champs audités :**
  - **USERNAME** : Identifiant de l'utilisateur.
  - **TIMESTAMP** : Date et heure de la connexion.
  - **ACTION_NAME** : Type d'action (par exemple, LOGON, LOGOFF).
  - **RETURNCODE** : Code de retour indiquant si la connexion a échoué

23

Audit des objets (tables, vues, procédures)

- **Exemple de commande :**
  - **AUDIT SELECT, INSERT, UPDATE, DELETE ON <schema>.<table>;**
- **But :** Suivre les actions effectuées sur des objets spécifiques dans la base de données.
- **Champs audités :**
  - **USERNAME** : L'utilisateur ayant exécuté l'action.
  - **OBJECT_NAME** : Le nom de l'objet (table, vue).
  - **ACTION_NAME** : L'action effectuée (SELECT, INSERT, DELETE).

24

Contrôle d'Audit des Privilèges

- politique d'audit basée sur les privilèges (Audit des privilèges utilisateurs)
- **Exemple de commande :**
  - AUDIT GRANT ANY PRIVILEGE;
- **But :** Suivre l'attribution des privilèges aux utilisateurs (par exemple, GRANT de privilèges DBA).

25

Surveillance des Échecs de Connexion

- Suivi des connexions échouées
- **Exemple de requête :**
  - SELECT USERNAME, TIMESTAMP, RETURNCODE FROM DBA_AUDIT_TRAIL WHERE ACTION_NAME = 'LOGON' AND RETURNCODE != 0;
  - **Cas courants de code RETURNCODE :**
    - **1017** : Mot de passe incorrect.
    - **28000** : Utilisateur inconnu.
    - **12224** : Compte verrouillé.

26

Désactivation d'une Politique d'Audit

- Désactivation d'une politique d'audit
  - **Commande pour désactiver une politique d'audit :**
    - NOAUDIT SESSION;
    - **Impact :** Cette commande arrête l'enregistrement des connexions et déconnexions des utilisateurs.

27

Création de la politique d'audit personnalisée

- **Syntaxe générale**

CREATE AUDIT POLICY <policy_name>
ACTION <action> [ON <object>]
WHEN <condition>;

  - **<policy_name>** : Nom de la politique d'audit que vous définissez.
  - **<action>** : L'action spécifique à auditer (par exemple : SELECT, INSERT, UPDATE, DELETE, LOGON, etc.).
  - **<object>** : L'objet sur lequel l'action est effectuée (table, vue, procédure, etc.).
  - **<condition>** : Condition qui spécifie quand l'audit doit se produire (par exemple : SUCCESSFUL, FAILED, ou des conditions spécifiques liées à l'utilisateur).

Exemple de politique d'audit personnalisée

- **Exemple 1 : Audit des connexions réussies des utilisateurs DBA**
  - **Objectif :** Suivre toutes les connexions réussies des utilisateurs ayant des privilèges DBA (par exemple, SYSDBA, DBA).
  - **CREATE AUDIT POLICY** audit_dba_logon

    ACTION LOGON

    WHEN 'USER IN ("SYSDBA", "DBA")'

    WHENEVER SUCCESSFUL;

- Cette politique audite **toutes les connexions réussies** des utilisateurs appartenant à la liste 'SYSDBA' et 'DBA'.
- Elle s'applique uniquement lorsque la connexion est réussie.

Exemple de politique d'audit personnalisée

- **Exemple 2 : Audit de l'accès à une table sensible**
- **Objectif :** Auditer toutes les tentatives de lecture et d'écriture (SELECT, INSERT, UPDATE, DELETE) sur une table appelée SENSITIVE_DATA.

```
CREATE AUDIT POLICY audit_sensitive_data
 ACTION SELECT, INSERT, UPDATE, DELETE
 ON SENSITIVE_DATA
 WHENEVER SUCCESSFUL;
```

- Cette politique suit toutes les actions (lecture, insertion, modification, suppression) sur la table SENSITIVE_DATA
- Elle enregistre ces actions uniquement si elles sont **réussies**.

- **1. Vérifier la politique d'audit**
  - **SELECT * FROM** DBA_AUDIT_POLICIES **WHERE** POLICY_NAME = 'AUDIT_SENSITIVE_DATA';
- **2. Consulter les résultats d'audit** : Les résultats de l'audit pour cette politique seront enregistrés dans la vue DBA_AUDIT_TRAIL. Vous pouvez exécuter une requête pour consulter les actions auditées sur la table SENSITIVE_DATA :

  SELECT USERNAME, ACTION_NAME, OBJ_NAME, TIMESTAMP, RETURNCODE

  FROM DBA_AUDIT_TRAIL

  WHERE OBJ_NAME = 'SENSITIVE_DATA' **AND** ACTION_NAME **IN** ('SELECT', 'INSERT', 'UPDATE', 'DELETE');
- **3. Vérification des politiques d'audit au niveau des objets** : Vous pouvez également consulter les politiques d'audit définies au niveau des objets (comme une table spécifique) en exécutant :
  - **SELECT * FROM** DBA_OBJ_AUDIT_OPTS **WHERE** OBJECT_NAME = 'SENSITIVE_DATA';

31

- **Exemple 3 : Audit des utilisateurs ayant des privilèges spécifiques**
- **Objectif :** Auditer l'utilisation de privilèges sensibles tels que ALTER, DROP sur des tables sensibles.
  - **CREATE AUDIT POLICY** audit_privileges_usage
  - **ACTION** ALTER, DROP
  - **ON** SENSITIVE_TABLE
  - **WHENEVER** SUCCESSFUL;
  - Cette politique enregistre les tentatives de **modification (ALTER)** ou **suppression (DROP)** sur la table SENSITIVE_TABLE.
  - Les actions sont auditées uniquement lorsqu'elles sont **réussies**.

32

Application et gestion des politiques

- **Appliquer une politique d'audit**
  - Une fois que vous avez créé une politique d'audit, vous devez l'activer pour qu'elle commence à auditer les actions définies.
    - **AUDIT POLICY <policy_name>;**
- **Exemple :**
  - AUDIT POLICY audit_dba_logon;
    - Cela active la politique audit_dba_logon et commence à suivre les connexions réussies des utilisateurs DBA.

33

- **Désactiver une politique d'audit**
  - Si vous souhaitez désactiver une politique d'audit, utilisez la commande suivante :
    - **NOAUDIT POLICY <policy_name>;**
- **Exemple :**
  - **NOAUDIT POLICY audit_dba_logon;**
    - Cela désactive la politique audit_dba_logon.

- **Supprimer une politique d'audit :** Pour supprimer une politique d'audit, utilisez la commande suivante :
  - **DROP AUDIT POLICY <policy_name>;**
  - **Exemple :**
    - **DROP AUDIT POLICY audit_dba_logon;**
      - Cela supprime la politique audit_dba_logon de la base de données.

Suivi des résultats d'audit

- Une fois les politiques d'audit activées, vous pouvez interroger les tables de suivi des événements d'audit pour consulter les résultats.
- **Exemple de requête pour consulter les connexions réussies :**
  - **SELECT** USERNAME, ACTION_NAME, OBJ_NAME, TIMESTAMP, RETURNCODE
  - **FROM** DBA_AUDIT_TRAIL
  - **WHERE** ACTION_NAME = 'LOGON' AND RETURNCODE = 0;

- L'audit des connexions est essentiel pour la sécurité et la conformité des bases de données Oracle.
- La configuration correcte de l'audit et la gestion des logs sont cruciales pour surveiller les activités des utilisateurs.
- Assurez-vous que le paramètre **audit_trail** est bien configuré avant de tester.

Introduction à Oracle Virtual Private Database (VPD)

Introduction

- **Oracle VPD (Virtual Private Database)** est une fonctionnalité qui permet de restreindre l'accès aux données au niveau de la ligne, en appliquant des politiques d'accès spécifiques à chaque utilisateur ou session.
- **Caractéristiques principales :**
  - Sécurise l'accès aux données de manière dynamique.
  - Applique des politiques au niveau des lignes de tables, en ajoutant une clause WHERE dynamique à chaque requête.
  - Permet de personnaliser l'accès en fonction de l'utilisateur ou de ses privilèges.

Pourquoi utiliser Oracle VPD ?

- **Avantages de VPD :**
  - **Sécurité améliorée :** Restreint l'accès aux données sensibles en fonction des utilisateurs.
  - **Contrôle granulaire :** Définition de politiques d'accès au niveau de la ligne pour chaque utilisateur.
  - **Simplicité :** Intégration transparente dans les applications sans nécessiter de modification du code de l'application.
  - **Flexible :** Application de politiques complexes selon les rôles, les sessions ou des critères dynamiques.

40

Architecture Oracle VPD

- **Politiques d'accès :** Les règles définissant comment les données sont accessibles.
- **Fonctions de politique :** Fonction PL/SQL qui génère la clause WHERE dynamique.
- **Modules de politique :** Associent une fonction de politique à une table ou une vue.
- **Sécurisation des données :** Chaque requête exécutée sur les tables ou vues protégées est automatiquement modifiée par la politique avant son exécution.

41

Définir une politique d'accès avec VPD

- **Créer une fonction de politique**
  - Une fonction de politique est une fonction PL/SQL qui génère dynamiquement la clause WHERE pour une requête.

```
CREATE OR REPLACE FUNCTION secure_data_policy (schema_name IN VARCHAR2, table_name IN VARCHAR2)
  RETURN VARCHAR2 IS
BEGIN
  -- Exemple d'application d'une politique : restreindre l'accès aux données d'un département
  IF USER = 'HR_USER' THEN
    RETURN 'department = '''HR''';
  ELSE
    RETURN '1=1';  -- Accès total pour tous les autres utilisateurs
  END IF;
END secure_data_policy;
```

  - Si l'utilisateur est HR_USER, l'accès aux données est restreint aux enregistrements où le département est 'HR'.
  - Pour les autres utilisateurs, l'accès est illimité.

42

Associer une politique à une table

- Après avoir créé la fonction de politique, vous devez l'associer à la table ou à la vue concernée.

```
BEGIN
  DBMS_RLS.ADD_POLICY (
    object_schema   => 'HR',
    object_name     => 'EMPLOYEES',
    policy_name     => 'secure_department_policy',
    function_schema => 'HR',
    policy_function => 'secure_data_policy',
    statement_types => 'SELECT, INSERT, UPDATE, DELETE',
    update_check    => TRUE
  );
END;
```

  - **ADD_POLICY** applique la fonction de politique **secure_data_policy** à la table **EMPLOYEES** dans le schéma **HR**.
  - La politique s'applique pour les actions SELECT, INSERT, UPDATE, et DELETE.
  - L'option **update_check** garantit que la politique est appliquée lors de la modification des données.

43

Exemple d'utilisation de VPD

- Soit la table EMPLOYEES dans laquelle chaque employé appartient à un département (par exemple, 'HR', 'SALES', etc.). La politique ***secure_data_policy*** permet à un utilisateur avec un rôle spécifique d'accéder uniquement aux employés de son département.
- Exemple de la **Table EMPLOYEES :**

EMPLOYEE_ID	NAME	DEPARTMENT
1	FRIHI Mohamed	HR
2	FERRAHI Ahmed	SALES
3	ZEMIRLI Hamid	HR
4	SID AHMED	SALES

- Exécution de la requête SELECT * FROM EMPLOYEES; par HR_user:

EMPLOYEE_ID	NAME	DEPARTMENT
1	FRIHI Mohamed	HR
3	ZEMIRLI Hamid	HR

44

Gestion et maintenance des politiques VPD

- **Supprimer une politique VPD** : Pour supprimer une politique d'audit appliquée à une table :

```
BEGIN
  DBMS_RLS.DROP_POLICY (
    object_schema => 'HR',
    object_name   => 'EMPLOYEES',
    policy_name   => 'secure_department_policy'
  );
END;
```

45

Cas d'usage de Oracle VPD

- **1. Sécurisation des données sensibles :**
  - Restreindre l'accès aux informations de paiement dans une table des employés.
- **2. Gestion de l'accès multi-tenants :**
  - Empêcher les utilisateurs d'un locataire d'accéder aux données d'un autre locataire dans une base de données multi-tenant.
- **3. Sécurisation des accès par rôle :**
  - Appliquer des restrictions sur les données accessibles par différents rôles ou départements.

46

- **Exemple de création d'une fonction de politique**
  - Certains utilisateurs ne puissent pas voir du tout la colonne B
  - 1. Création d'une fonction de politique de sécurité

```
CREATE OR REPLACE FUNCTION masquer_b(
    object_schema VARCHAR2,
    object_name VARCHAR2
) RETURN VARCHAR2 AS
BEGIN
    RETURN 'B = ''**''';
END;
/
```

47

- **Exemple de création d'une fonction de politique**
  - Certains utilisateurs ne puissent pas voir du tout la colonne B
  - 2. Appliquer la politique à la table T

```
BEGIN
    DBMS_RLS.ADD_POLICY(
        object_schema   => 'USER_NAME',
        object_name     => 'T',
        policy_name     => 'Masquer_B_Policy',
        function_schema => 'USER_NAME',
        policy_function => 'masquer_b',
        statement_types => 'SELECT'
    );
END;
/
```

48

Chapitre III Dictionnaire de données

Introduction

- Une base de données contient des relations de base, des chemins d'accès et des vues
- Le SGBD, pour fonctionner et assurer ses objectifs doit pouvoir retrouver les différentes informations sur ces catégories d'information.
- CATALOGUES Les catalogues sont des relations (tables).
  - Ces relations contiennent des informations sur tous les objets de la base de données .
  - Sans ces informations le SGBD ne peut pas assurer ses objectifs.

PRINCIPAUX CATALOGUES

- **La relation RELATION :** Chacun de ses tuples décrit une relation de la base.
- **La relation ATTRIBUT:** chacun de ses tuples décrit un attribut d'une relation de la base
- **La relation INDEX:** chacun de ses tuples décrit une relation index crée sur une relation de base ou système.
- **La relation VUE:** chacun de ses tuples décrit une vue de la base.
- **La relation AUTORISATION:** chacun de ses tuples décrit un utilisateur avec les droits qui lui sont accordés sur chaque objet de la base(lecteur/écriture/modification…)

Relation

- Chaque tuple est un descripteur d'une relation dans la base de données.
- Informations incluses :
  - Nom de la relation
  - Type de la relation
  - Taille ▯ Cardinalité
  - Date de création,
  - Adresse du premier attribut

Nom relation (nom externe)	Type relation	Ident relation (nom interne)	Taille tuple (nbre octets)	Cardinalité (nombre tuples)	Degré (nombre attributs)	Date création	Version	Adresse 1er attribut
Relation	système	1	100	50	09	25/03/02	1	@
Fournisseur	Base	10	110	1000	04	25/03/02	1	@

52

ATTRIBUT

- Chaque tuple décrit un attribut d'une relation de la base
- Informations incluses :
  - Nom de l'attribut
  - Identificateur attribut
  - Identificateur relation
  - Type attribut
  - longueur
  - Position de l'attribut dans la table

Nom attribut (nom externe)	Identificateur attribut	Identificateur relation	Type attribut	longueur	Position attribut dans la relation
Numf	11	02	Integer	06	01
Nomf	12	02	Char	10	02

53

Index

- Chaque tuple décrit UN index cré sur une relation de base ou système
- Informations incluses :
  - Nom de l'index
  - Identificateur de la relation indexée
  - Type de l'index
  - Attributs indexés

Nom index	Identification relation	Type (primaire , secondaire)	Liste arguments index
XFOUR	**02**	**P**	**Numf**
XSFOUR	**02**	**S**	**villef**

54

Vue

- Chaque tuple décrit une vue créée sur la base.
- Informations incluses :
  - Identificateur de la vue
  - Identificateurs des relations de base utilisées par la vue
  - Requête définissant la vue

Identificateur(s) des relations de base	Identificateur de la vue	Requête qui définit la vue
02	100	Select * from fournisseur where villef ='Tunis'

55

Catalogue : SGBD

- Les catalogues dépendent du SGBD
- Chaque SGBD possède une spécificité
- D'autres catalogues peuvent exister
- Les structures des catalogues peuvent être différentes selon les SGBDs.

56

Oracle

57

LE DICTIONNAIRE DE DONNÉES ORACLE

- Organisé comme une base de données (ensemble de tables)
- La plupart de ses tables ne sont pas modifiables par les utilisateurs(seul l'administrateur de la base système, a accès à ces dernières) .
- Le contenu de ces tables est accessible par l'intermédiaire des vues.

58

CLASSEMENT DE VUES DE DICTIONNAIRE DE DONNÉES

- **USER_XXX**
  - décrit les objets appartenant à l'utilisateur connecté
- **ALL_XXX :**
  - décrit les objets accessibles à tous les utilisateurs
- **DBA_XXX :**
  - décrit tous les objets (vues autorisées aux administrateurs de la base seulement…)
- **V$_XXX :**
  - pour le suivi des performances

59

QUELQUES VUES DE DICTIONNAIRE D'ORACLE

Nom de la vue	contenu
DICTIONARY (DICT)	Vues permettant de consulter le dictionnaire de données
USER_TABLES (TABS)	tables et vues crées par l'utilisateur
ALL_CATALOG (CAT)	Tables et vues sur lesquelles l'utilisateur a des droits (lecture, écriture, modification …)
USER_TAB_COLUMNS (COLS)	Colonnes de chaque table ou vue crée par l'utilisateur courant
USER_VIEWS	Vues crées par l'utilisateur
USER_INDEXES (IND)	Indexes crées par l'utilisateur ou indexant des tables crées par l'utilisateur
USER_TAB_PRIVS	Objets sur lesquels l'utilisateur est propriétaire, donneur ou receveur d'autorisation
USER_CONSTRAINTS	Définition de contraintes pour les tables d'utilisateur

60

REQUÊTES UTILES

- **DESCRIBE table/vue**
  - pour avoir tous les attributs de cette table ou vue
  - **SELECT comments**
    FROM DICT
    WHERE table_name='nom d'une vue de dictionnaire de données'
- Utiliser l'attribut **comments** de la vue **DICTIONARY** pour connaitre le contenu de la vue.

61

QUELQUES EXEMPLES

- **Structure d'une table**

```
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH, DATA_PRECISION  FROM USER_TAB_COLUMNS WHERE TABLE_NAME =
'INSTALLER';

COLUMN_NAME          DATA_TYPE                 DATA_LENGTH          DATA_PRECISION
-----------  ---------------------------  ---------  ------------------------------------------------
NPOSTE               VARCHAR2                  7
NLOG                 VARCHAR2                  5
NUMINS               NUMBER                    22                   5
DATEINS              DATE                      7
DELAI                INTERVAL DAY(5) TO SECOND(2)  11               5
```

62

QUELQUES EXEMPLES

- **Tous les Objets d'un schéma**

```
SELECT OBJECT_NAME, OBJECT_TYPE, CREATED FROM USER_OBJECTS;

OBJECT_NAME              OBJECT_TYPE          CREATED
-------------------------  -------------------  -----  -----  -----  -----
ACCES_SECURISE           PACKAGE              03/09/03
ACCES_SECURISE           PACKAGE BODY         03/09/03
AFFICHEAVIONS            PROCEDURE            03/09/03
Compagnies               JAVA CLASS           17/08/03
EFFECTIFSHEURE           FUNCTION             16/09/03
ESPIONCONNEXION          TRIGGER      16/09/03
PILOTE                   TABLE        18/09/03
PK_PILOTE                INDEX        18/09/03
VUEMULTICOMPPIL          VIEW         14/09/03
```

63

SQL SERVER

LES CATALOGUES SYSTÈME DE SQL SERVER

- Un ensemble de vues affichant les métadonnées décrivant les objets dans une instance de SQL Server
- Les métadonnées ne sont visibles que pour les éléments sécurisables qu'un utilisateur détient ou pour lesquels des autorisations lui ont été accordées.

L'ACCÈS AUX INFORMATIONS DES CATALOGUES SYSTÈME

- **Affichage catalogue.** (la plus recommandée)
- Vues des schémas d'information.
- Ensembles de lignes de schémas OLE DB.
- Fonctions de catalogue ODBC.
- Fonctions et procédures stockées système.

VUES D'AFFICHAGE CATALOGUE OBJETS

Vue de catalogue	Contenu
Sys.objects	Contient une ligne pour chaque objet(table sys, table user, contraint check, primary key..)
Sys.tables(hérite de sys.objects	Toutes les table de la base en cours (définies par l'utilisateur)
sys.columns, sys.computed_columns(calculée) sys.foreign_key_column sys.identity_columns	informations sur toutes les colonnes pour chaque table de la base, les autres vues sont pour des colonnes de type particulier (elles héritent de sys.columns)
sys.foreign_keys sys.foreign_key_columns	Informations sur les contraintes de type clé étrangères
sys.indexes	Information sur les indexes de tables de la base en cours(nom de l'index, la table sur laquelle l'index est crée)
sys.check_constraints, sys.default_constraints, sys.key_constraints	Informations sur les différentes contraintes .

67

EXEMPLE DE REQUÊTES

- Toutes les tables définies par l'utilisateur dans une base de données spécifiée(simple):

```
Use
go SELECT * FROM sys.tables
go
```

68

MySQL

69

MySQL

- MySQL dispose d'un catalogue système pour l'ensemble des bases de données présentes dans le SGBD.
- Le nom de cette base, accessible en lecture, est **INFORMATION_SCHEMA**.
- Une collection de vues systèmes et non pas de tables physiques.
- Pour se placer dans le catalogue, il suffit de taper la commande suivante :

  USE INFORMATION_SCHEMA ;

CATALOGUES ET DICTIONNAIRE DE DONNÉES

- Les informations concernant les bases sont décrites dans la vue système **SCHEMATA**.
- Pour obtenir toutes les informations sur une base de données on utilise l'attribut **SCHEMA_NAME.**
- **SELECT** *

  FROM INFORMATION_SCHEMA. SCHEMATA

  WHERE SCHEMA _NAME = 'nom _base';

LES TABLES

- Les métadonnées des tables sont présentes dans la vue système TABLES. Les informations couramment utilisées dans cette vue système sont :
  - TABLE_SCHEMA
  - TABLE_NAME
  - TABLE_TYPE
  - ENGINE
  - TABLE_ROWS
  - DATA_LENGTH
  - MAX_DATA_LENGTH
  - INDEX_LENGTH
  - AUTO_INCREMENT
  - CREATE_TIME : date et heure de création de la table ;
  - TABLE_COLLATION

LES COLONNES

- Les métadonnées sur les colonnes (ou champs) sont contenues dans la vue système COLUMNS.
- Les informations les plus utiles dans cette vue système sont les suivantes :
  - **TABLE_SCHEMA**
  - **TABLE_NAME**
  - **COLUMN_NAME**
  - **ORDINAL_POSITION**
  - **COLUMN_DEFAULT : la valeur par défaut de la colonne ;**
  - **IS_NULLABLE DATA_TYPE**
  - **CHARACTER_MAXIMUM_LENGTH**
  - Etc…..

LES INDEX

- La vue système **STATISTICS** contient les métadonnées sur les index
- Une partie de cette information se retrouve dans la vue système décrivant les colonnes.
- Les informations les plus utiles sont:
  - TABLE_SCHEMA
  - TABLE_NAME
  - NON_UNIQUE
  - INDEX_NAME
  - SEQ_IN_INDEX
  - COLUMN_NAME
  - CARDINALITY
  - INDEX_TYPE

LES CONTRAINTES

- Pour les informations sur les contraintes d'une table, il faut utiliser la combinaison de deux nouvelles vues systèmes de métadonnées :
  - **TABLE_CONSTRAINTS et KEY_COLUMN_USAGE.**
- La première des deux vues fournit principalement le type de la contrainte, alors que la seconde donne les tables et colonnes qui utilisent cette contrainte

EXEMPLE

- Afficher la liste des clés étrangères d'une table
  - **SELECT** *
    FROM INFORMATION _SCHEMA.**TABLE _CONSTRAINTS**
    WHERE TABLE _SCHEMA = 'nom _base' **AND**
    TABLE _NAME = 'nom _table' **AND**
    CONSTRAINT _TYPE = 'FOREIGN KEY';

LES PROCÉDURES STOCKÉES, FONCTIONS ET DÉCLENCHEURS

- Utilisation de la vue système ROUTINES
- **Exemple**
  - Les informations sur les procédures stockées et fonctions (routines **SELECT** *
    FROM INFORMATION_SCHEMA.**ROUTINES**

LES TRIGGERS :

- Le catalogue fournit, depuis la version 5.0.10 de MySQL, des informations sur les déclencheurs (triggers).
- Quelques données contenues dans le dictionnaire
  - TRIGGER_NAME
  - EVENT_MANIPULATION
  - EVENT_OBJECT_SCHEMA
  - EVENT_OBJECT_TABLE
  - ACTION_ORDER
  - ACTION_STATEMENT
  - ACTION_ORIENTATION
  - ACTION_TIMING • DEFINER

LES PRIVILEGES

- Quatre vues systèmes fournissent des informations sur les privilèges globaux sur un serveur MySQL
  - **USER_PRIVILEGES ;**
  - **SCHEMA_PRIVILEGES ;**
  - **TABLE_PRIVILEGES ;**
  - **COLUMN_PRIVILEGES.**

- La vue système USER_PRIVILEGES fournit des informations sur les utilisateurs déclarés. Ces informations sont issues de la table MYSQL.USER